Workforce0 — Open Source AI Workforce

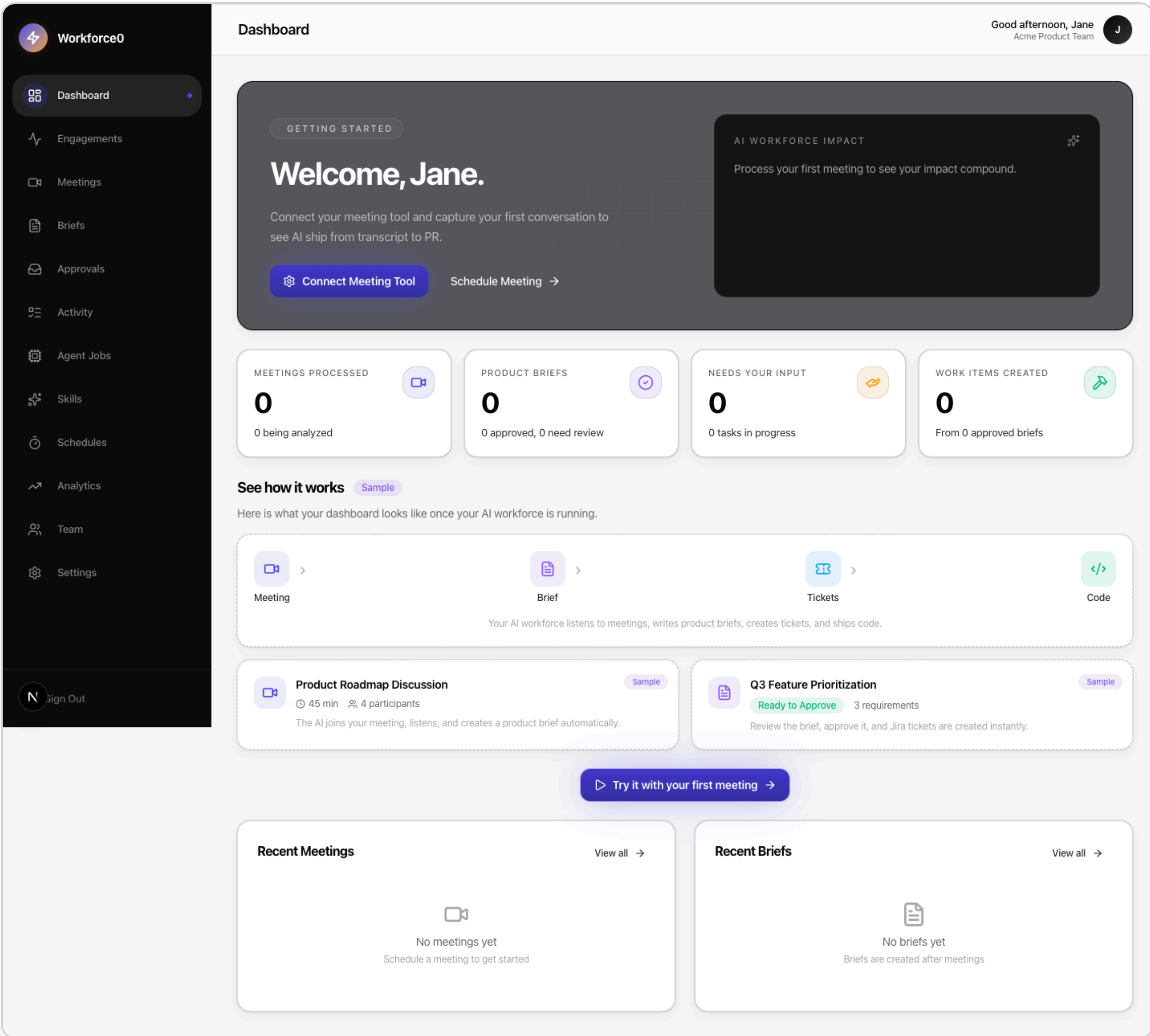
Workforce0

Meetings go in. Shipped work comes out.

Open-source, self-hosted AI workforce for product teams. Capture any conversation, get structured briefs, approve what ships, and have pull requests opened on your own repos — all running on your infrastructure with your own AI keys.

License MIT | Node 20+ | TypeScript 5.6 | Next.js 16 | Fastify 5 | PRs welcome

What it does · Screenshots · Quick start · Development · Architecture · BYOK · Contributing



What it does

Workforce0 is **AI that works the way your product team already works** — meetings, briefs, approvals, tickets — but everything in between is automated.

You capture a conversation. Workforce0 turns it into a structured brief with objectives, requirements, acceptance criteria, risks, and an architectural design. You review and approve. Approved briefs auto-generate Jira tickets (or Linear, or Notion pages). An optional local agent opens pull requests on your GitHub repos using your own Claude Code / Cursor / Codex subscription.

Every step has a human approval gate. Nothing ships without your click.

The end-to-end loop

```
┌─────────────┐    ┌──────────┐    ┌──────────────┐    ┌────────────┐
┌──────────────┐    ┌─────────┐
│   Meeting   │───▶│    AI    │───▶│    Human     │───▶│ Jira/Linear│───▶│ Local agent
│───▶│   PR    │
│             │    │  brief   │    │   approval   │    │   ticket   │    │  (BYO CLI)
│     │        │
│ upload/meet │    │  ~20 s   │    │              │    │            │    │
│     │        │
│ /phone/vtt  │    │          │    │              │    │            │    │
│     │        │
└─────────────┘    └──────────┘    └──────────────┘    └────────────┘
└──────────────┘    └─────────┘
```

Who it's for

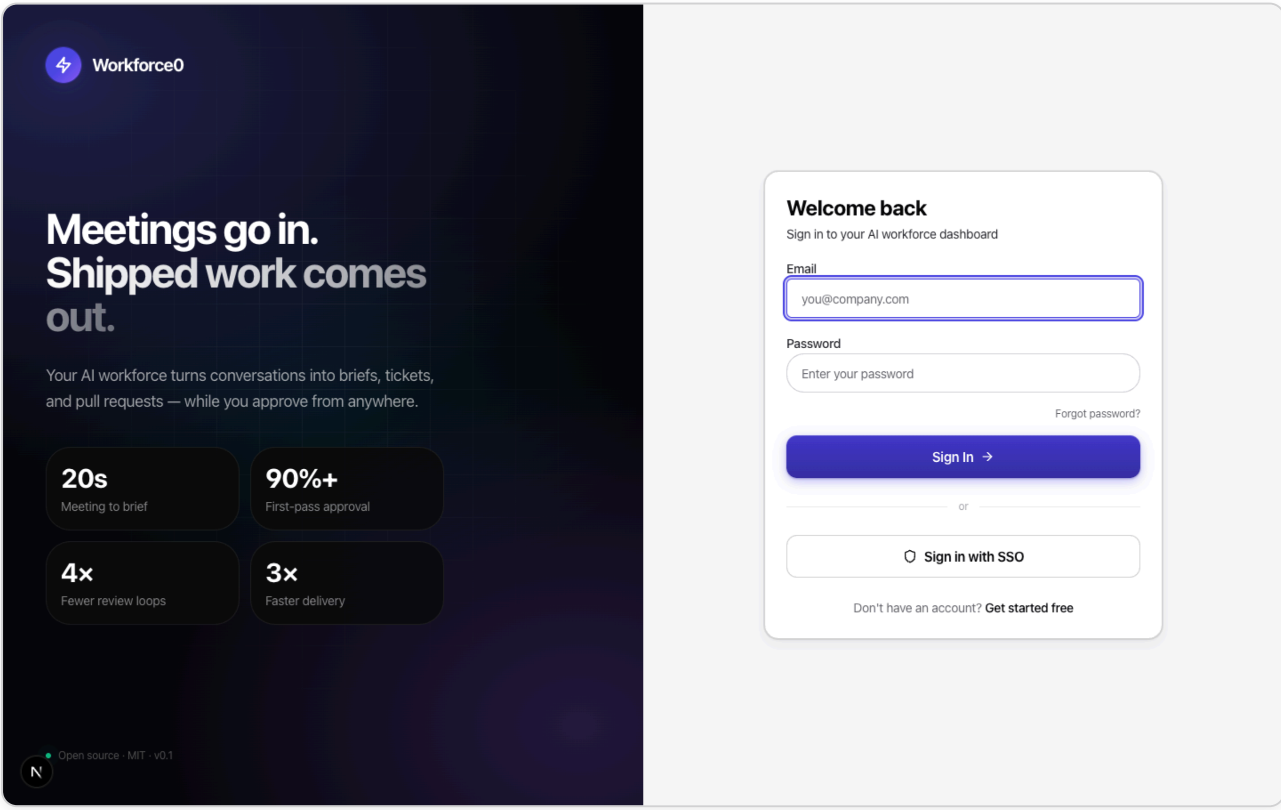
Non-technical product leaders — VPs, CPOs, operators. The UI walks you through everything in plain language. The CLI is optional. A technical friend can handle the initial deploy; after that, you live in the browser.

Why self-host

- **Your data, your machines.** Meeting transcripts never leave your infrastructure.
- **Your AI keys, your costs.** Bring your own Anthropic / Gemini / OpenAI keys — no per-token markup, no per-seat SaaS.
- **Your integrations, your rules.** Jira, Slack, GitHub, Google Drive, Teams — all wired to your own accounts.
- **No vendor lock-in.** MIT-licensed. Fork it, extend it, ship it.

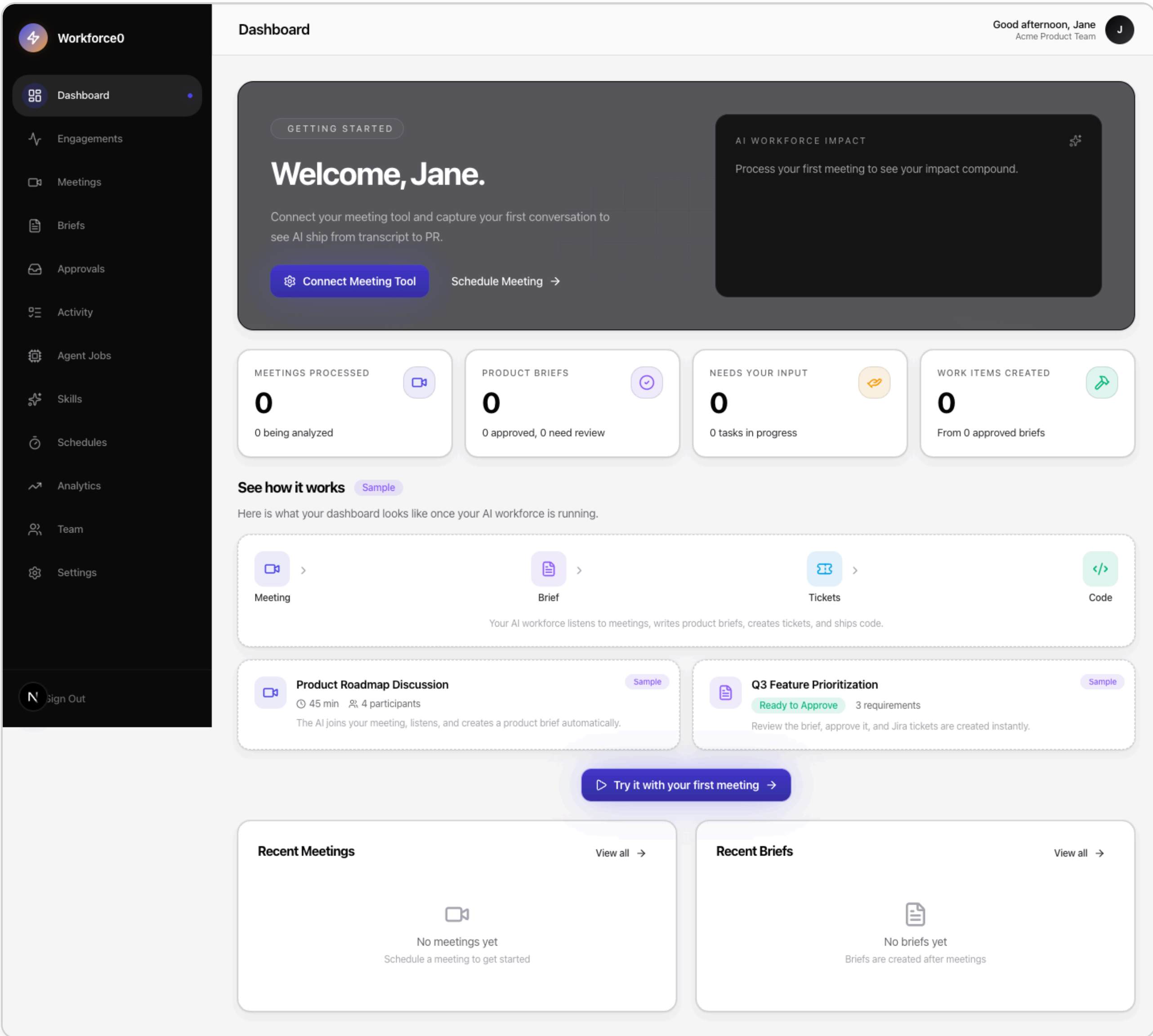
Screenshots

Sign-in



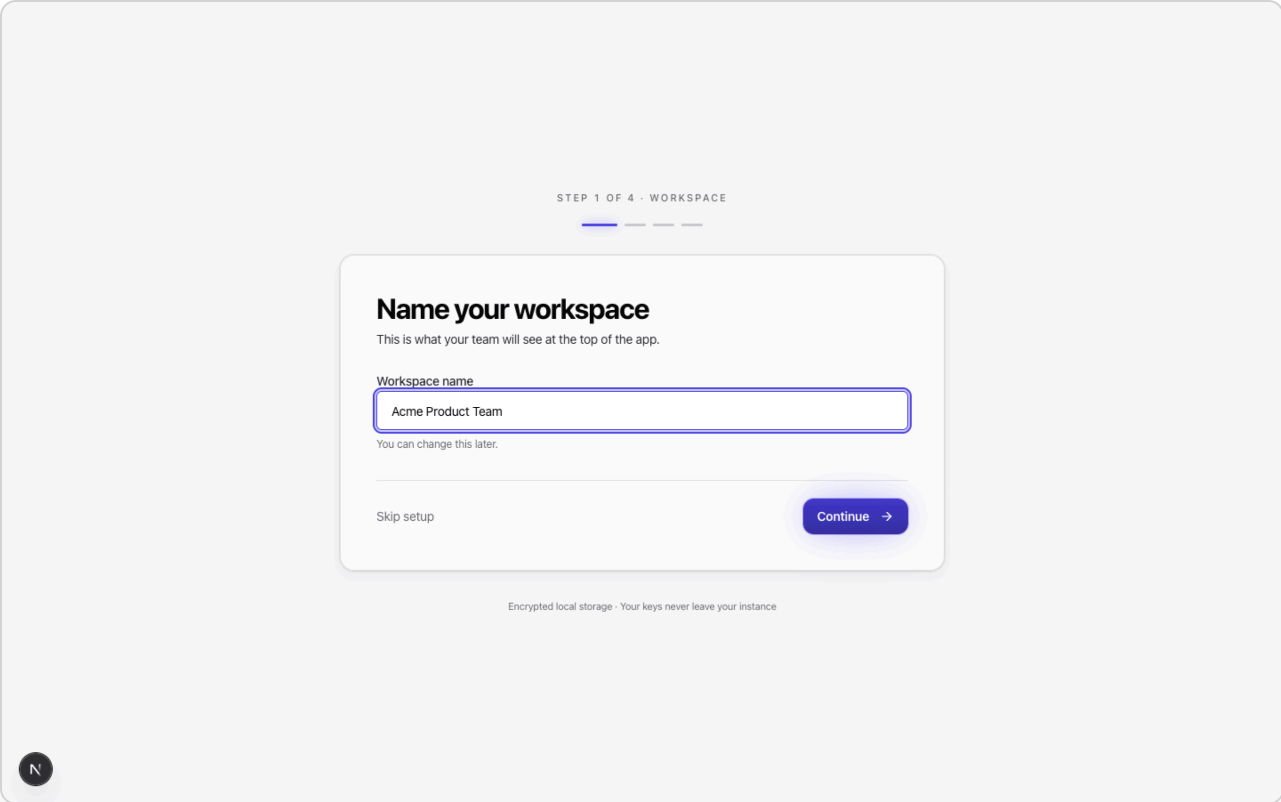
Cinematic sign-in shell with mesh gradient + display typography. Passwordless SSO optional.

Dashboard



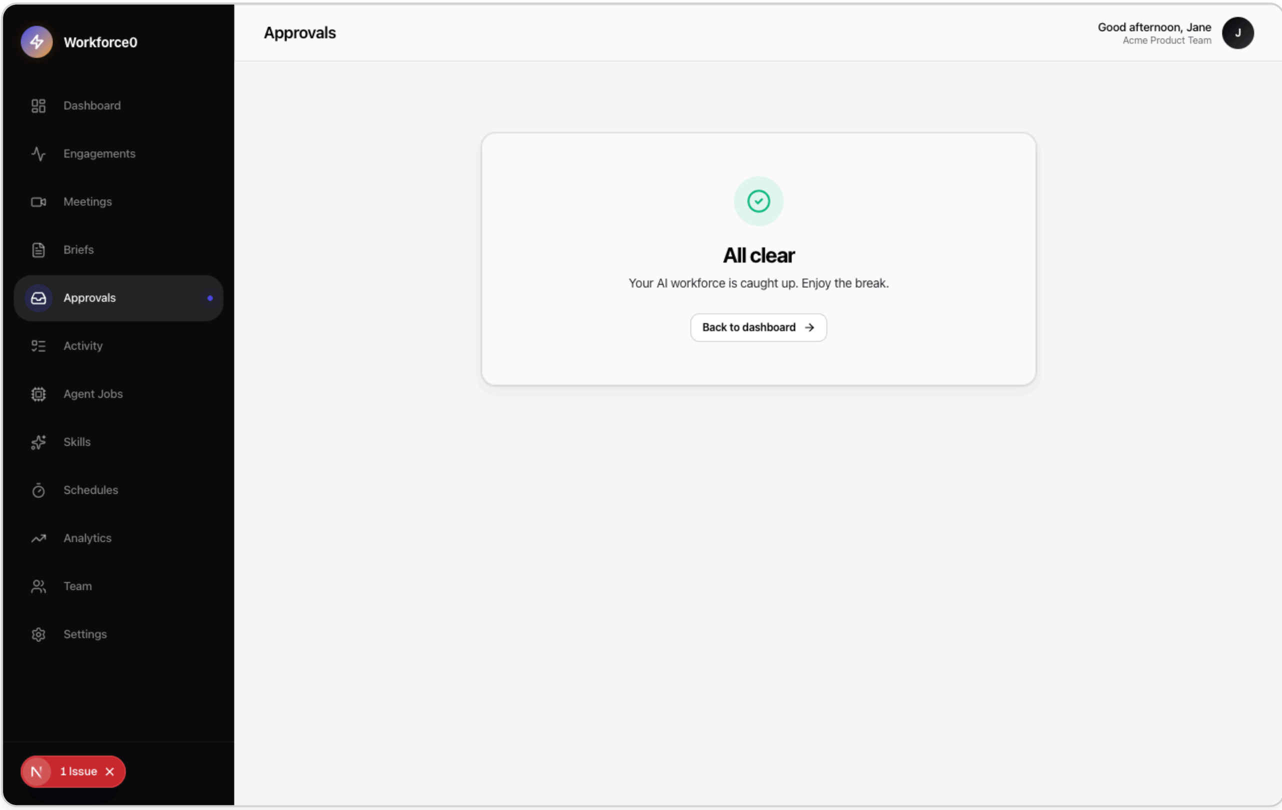
Hero banner with live AI-workforce impact panel, four tabular stat cards, sample preview for new users, recent activity feed.

Four-step setup wizard



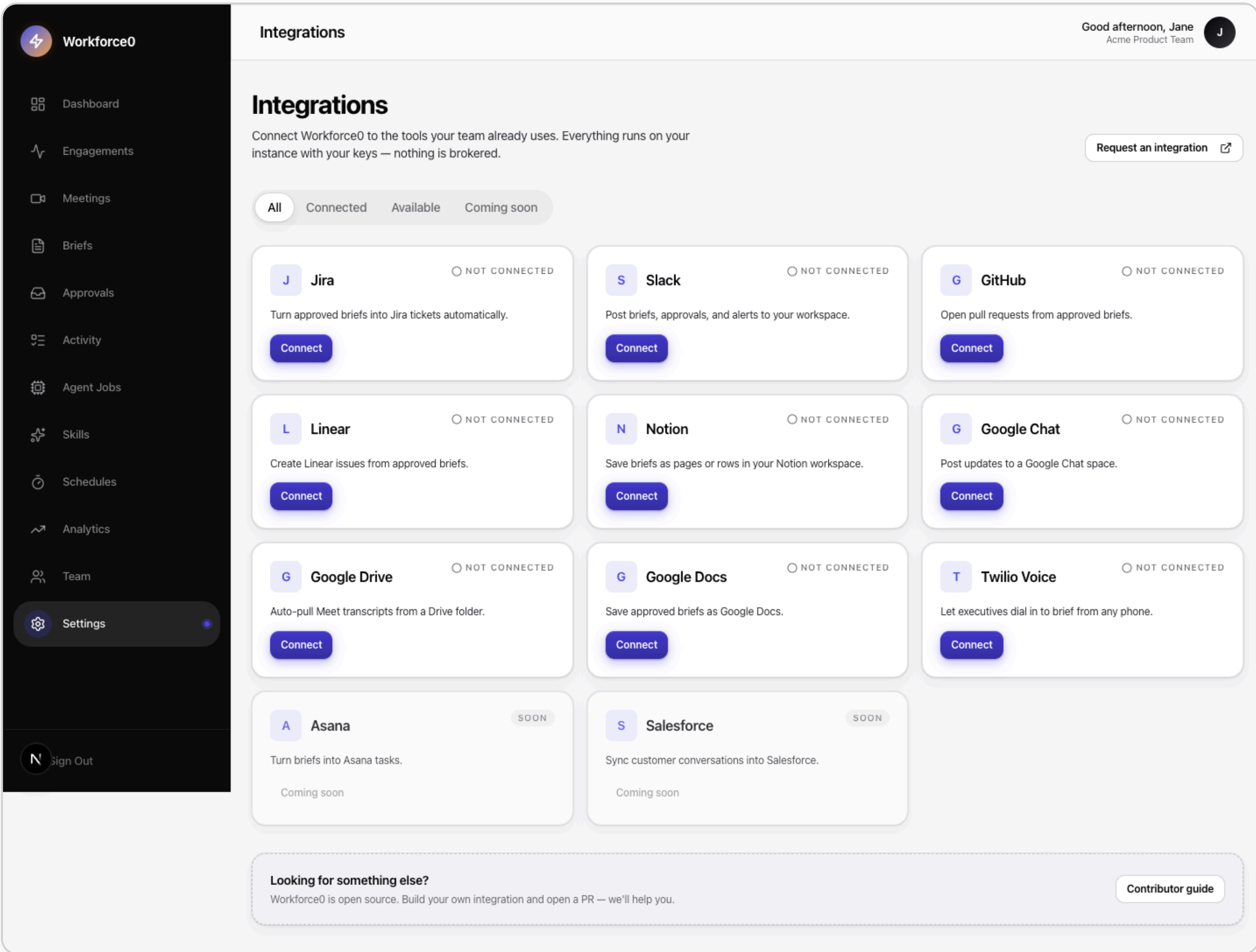
From fresh install to your first brief in under five minutes — workspace name, AI key, one integration, done.

Approvals queue



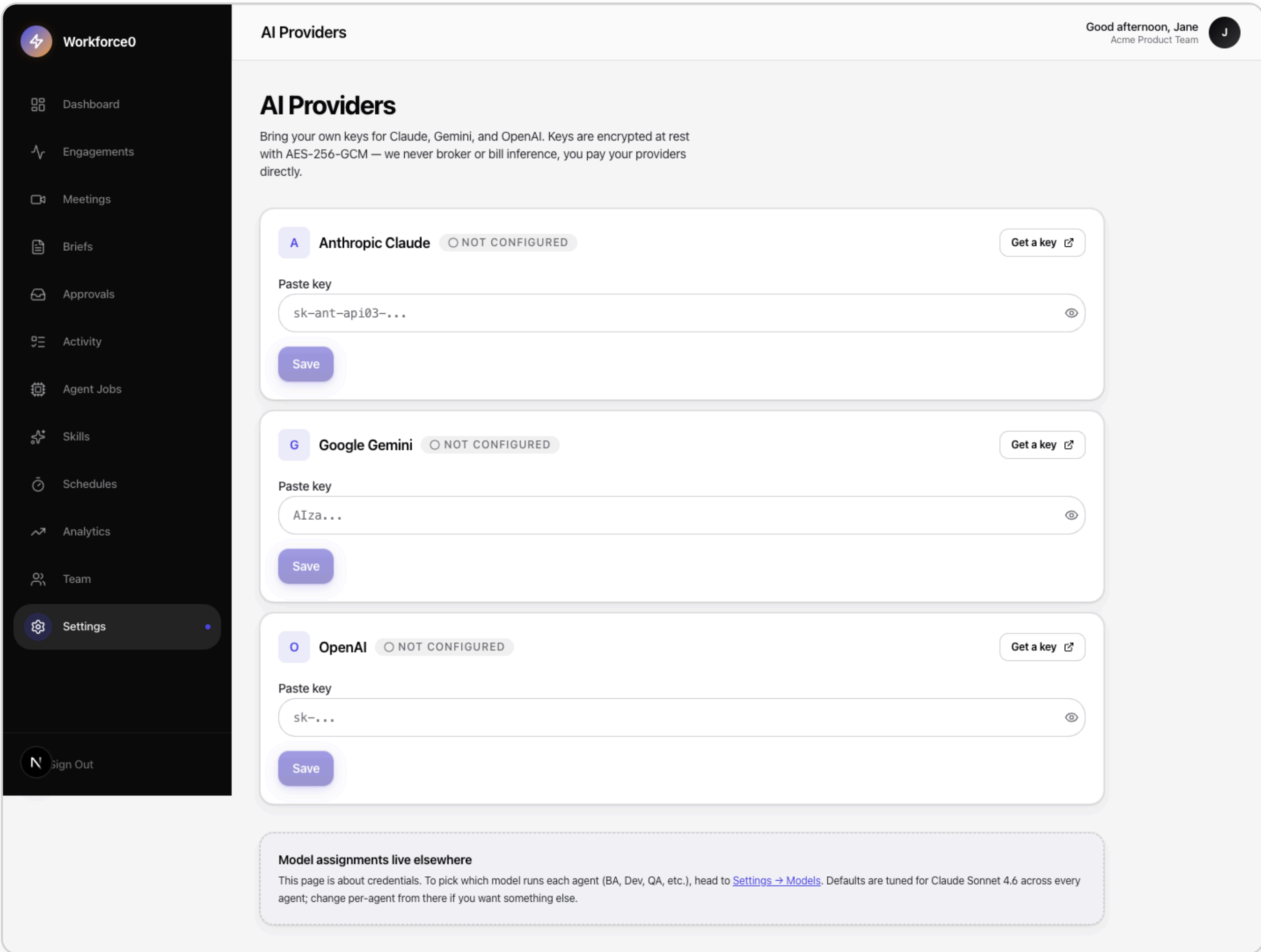
Linear-inspired two-pane inbox for triaging briefs, tickets, and PRs. `J/K` to navigate, `A` to approve, `R` to reject.

Integrations grid



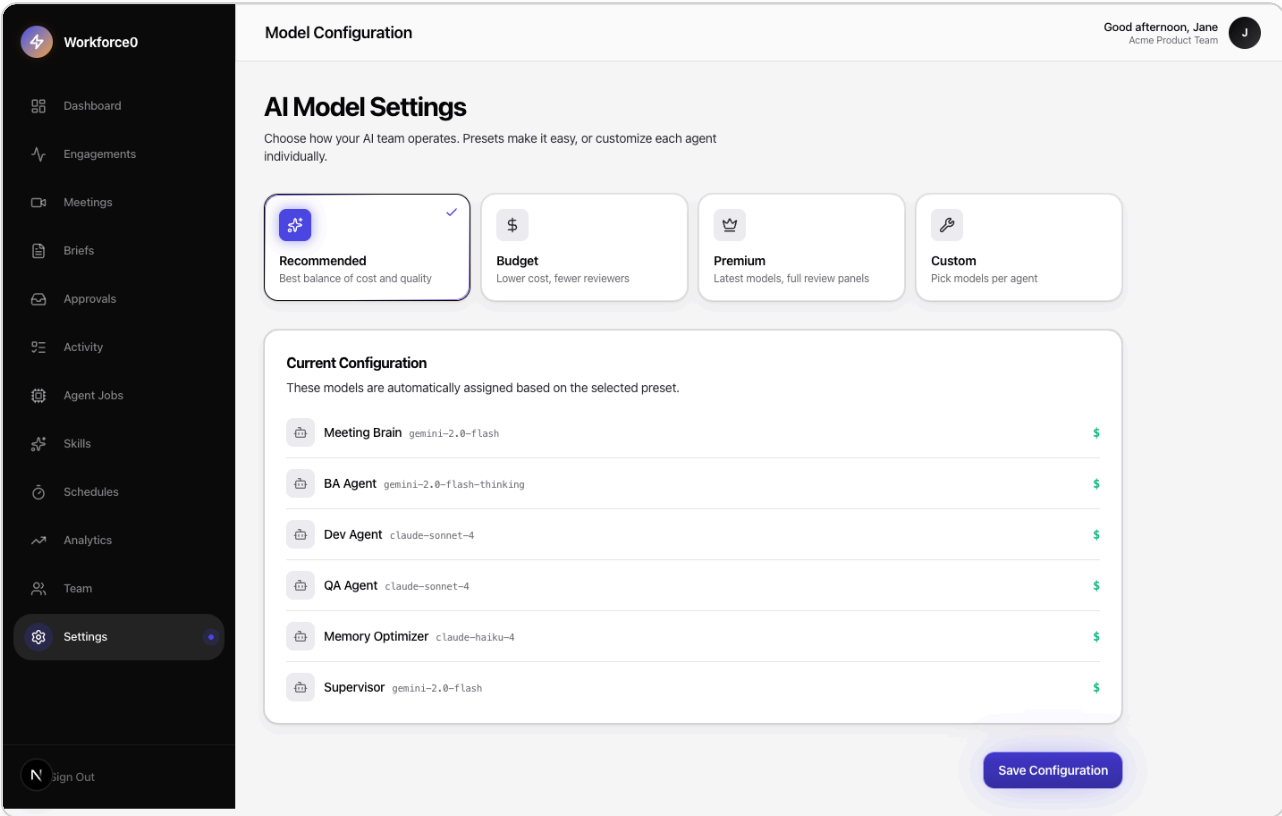
Jira, Slack, GitHub, Linear, Notion, Google Chat, Google Drive, Docs, Twilio. All via in-app wizards — no `.env` editing.

AI providers (BYOK)



Paste keys for Claude, Gemini, or OpenAI. Stored AES-256-GCM encrypted. Test-connection button per provider.

Model presets

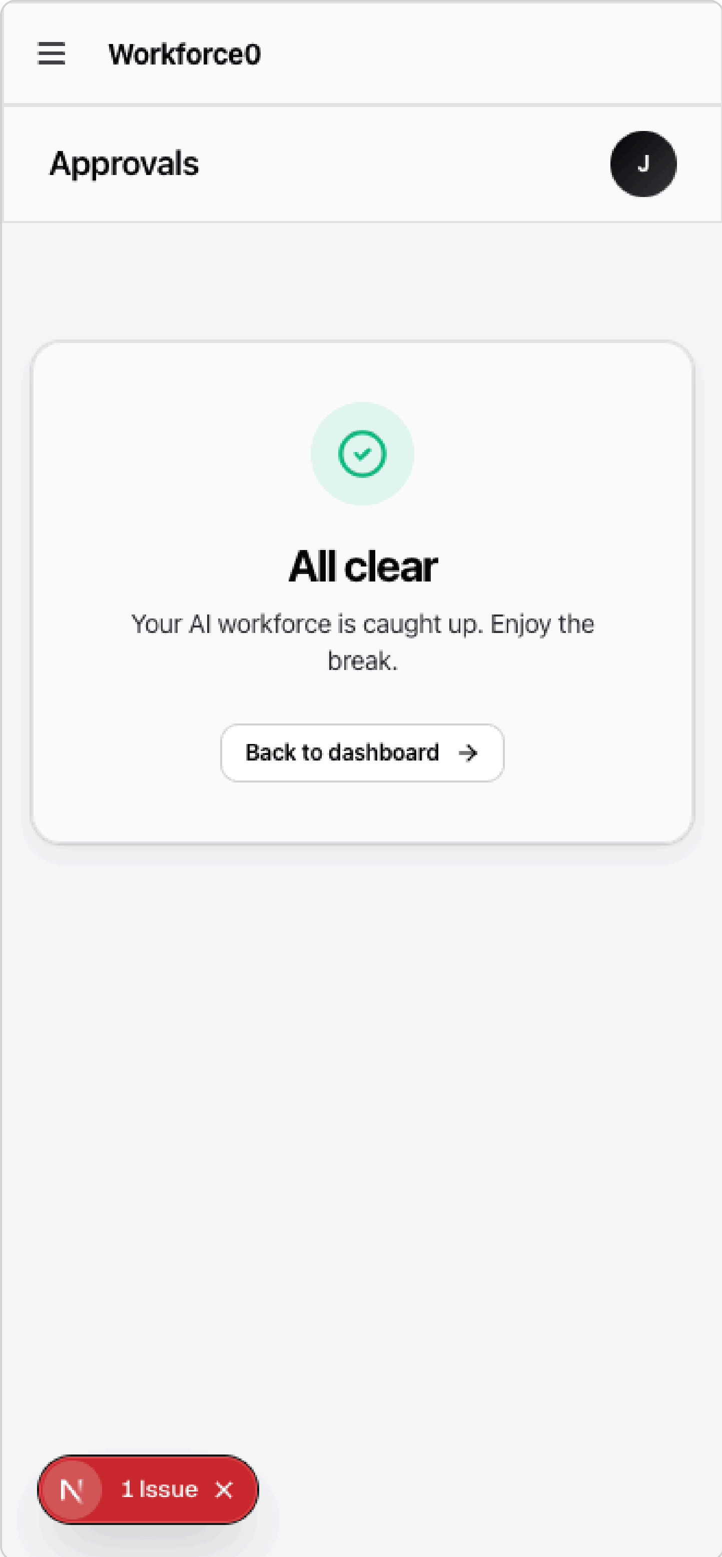


Pick Recommended / Budget / Premium presets or wire a custom model to every agent (BA, Dev, QA, Supervisor, Memory, Meeting Brain).

▶ **More screens**

Approvals on the phone

The approval queue is responsive — two-pane on the desktop, single-pane + fixed bottom action bar on mobile. And for the truly mobile workflow: reply `APPROVE <token>` or `REJECT <token>` to the Workforce0 WhatsApp/SMS/Slack message and the decision lands server-side without opening the app.


Workforce0
Approvals
J
All clear
Your AI workforce is caught up. Enjoy the break.
Back to dashboard
N
1 Issue

Quick start

Requirements

- **Docker** 20+ (or Node 20 + Postgres 15 + Redis 7 locally)
- **4 GB RAM** minimum
- **A Gemini API key** — free tier works

One command (Docker Compose)

```
git clone https://github.com/ithadmin/workspace0.git workforce0
cd workforce0
cp .env.example .env
# open .env and paste your GEMINI_API_KEY + a 32-char JWT_SECRET
docker compose -f docker-compose.prod.yml up -d
```

Then open **http://localhost:3001** and create your account. The in-app setup wizard handles the rest.

From clone to first brief: under five minutes.

One-click deploy

PLATFORM	DEPLOY	FREE TIER
Railway	Template →	✅
Render	Blueprint →	✅
Fly.io	`docs/one-click-deploy.md`	✅
DigitalOcean	Marketplace →	💰

Development

For contributors and anyone who wants to run it outside Docker.

1. Clone and install

```
git clone https://github.com/ithadmin/workspace0.git workforce0
cd workforce0

# Install backend deps
cd backend && npm install && cd ..

# Install frontend deps
cd frontend && npm install && cd ..
```

2. Start infrastructure

```
# Starts postgres:5432 + redis:6379
docker compose up -d postgres redis
```

Or use your own Postgres/Redis and set connection strings in `backend/.env`.

3. Configure the backend

```
cp backend/.env.example backend/.env
```

Edit `backend/.env` and set at minimum:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/workforce0
REDIS_URL=redis://localhost:6379/0
JWT_SECRET=<a-32-character-or-longer-secret>
GEMINI_API_KEY=<your-key>   # or configure via the UI later
```

4. Set up the database

```
cd backend
npx prisma db push       # sync schema
npx prisma generate      # generate client
```

5. Start both dev servers

In two terminals:

```
# Terminal 1 — backend (http://localhost:3000)
cd backend
npm run dev

# Terminal 2 — frontend (http://localhost:3001)
cd frontend
NEXT_PUBLIC_API_URL=http://localhost:3000 npm run dev
```

Open http://localhost:3001 and sign up.

Useful scripts

COMMAND	WHAT IT DOES
`cd backend && npm run dev`	Backend with tsx watch, hot reload
`cd backend && npm test`	1,200+ unit + integration tests (vitest)
`cd backend && npm run db:studio`	Prisma Studio — inspect the database
`cd backend && npm run db:migrate`	Create + apply a migration
`cd frontend && npm run dev`	Next.js dev server (port 3001)
`cd frontend && npm run typecheck`	TypeScript strict check
`cd frontend && npm run test:e2e`	Playwright E2E smoke tests
`cd agent && npm run dev`	Local agent daemon (opens PRs via your CLI)

Running on different ports

```
# Backend
cd backend && PORT=7777 npm run dev

# Frontend (points at backend)
cd frontend && NEXT_PUBLIC_API_URL=http://localhost:7777 \
  npx next dev --port 7778
```

Architecture

```
flowchart TB
  subgraph "Browser (exec-friendly UI)"
    FE[Next.js 16 + Radix + Tailwind v4]
  end

  subgraph "Backend API (Fastify)"
    API[REST + SSE + webhooks]
    MREG[ModelRegistry<br/>single point for AI calls]
    BA[BA Agent — brief generation]
    ARCH[Architect — design from brief]
    DEV[Dev tools — read/write PRD, code tools]
    QA[QA Agent — test + review]
    MEM[Honcho memory — grows with you]
  end

  subgraph "Infra"
    PG[(Postgres — tenant data<br/>BYOK encrypted AES-256-GCM)]
    RE[(Redis — sessions, queues, STT buffer)]
  end

  subgraph "Your AI providers (BYOK)"
    GEM[Gemini]
    ANT[Claude]
    OAI[OpenAI]
  end

  subgraph "Your integrations"
    JIRA[Jira / Linear / Notion]
    COMM[Slack / Google Chat / Teams]
    VCS[GitHub]
    TWIL[Twilio — dial-in voice]
    GMEET[Google Meet — auto-ingest]
  end

  subgraph "Your machine"
    AGENT[Local agent daemon<br/>uses YOUR Claude Code / Cursor subscription]
  end

  FE <--> API
  API <--> PG
  API <--> RE
  API --> BA --> MREG
  API --> ARCH --> MREG
  API --> QA --> MREG
  MREG --> GEM
  MREG --> ANT
  MREG --> OAI
  BA <--> MEM
  API --> JIRA
  API --> COMM
  API --> GMEET
  API <-. WebSocket .-> AGENT
  AGENT --> VCS
  TWIL --> API
```

Key design choices

- **Executive-first UI, not dev-first.** The polished Next.js frontend is the primary surface. The CLI is optional after initial deploy.
- **All AI calls go through `ModelRegistryService`.** Product code never imports provider SDKs directly. Swap Gemini for Claude for GPT without touching any agent.
- **Graceful degradation.** Any integration you haven't configured disables cleanly. No crashes, no angry errors.
- **Single-org by default.** Row-level `tenantId` exists but defaults to a single workspace for self-hosted installs. Multi-tenant SaaS is possible but not the focus.
- **AI Council (optional).** Multi-model consensus: Gemini drafts, GPT-o1 critiques, Claude adjudicates. Off by default (faster, cheaper). Enable with `AI_COUNCIL_ENABLED=true`.

Bring your own keys (BYOK)

Workforce0 never holds AI credentials or bills you for inference. You provide keys for the providers you already pay for:

PROVIDER	PURPOSE	HOW TO GET A KEY
Google Gemini	Brief generation, transcription	aistudio.google.com/apikey — free tier
Anthropic Claude	Critique, long-context reasoning	console.anthropic.com
OpenAI	GPT-o1 adversarial critique	platform.openai.com

Keys are stored **AES-256-GCM encrypted at rest** and used only by your instance. Configure them via **Settings → AI Providers** or set them once in `backend/.env`:

```
GEMINI_API_KEY=AIza...          # required
ANTHROPIC_API_KEY=sk-ant-...    # optional
OPENAI_API_KEY=sk-...           # optional
```

Full provider list and setup walkthroughs: `docs/byok.md`.

Features

Meetings

- **Upload.** `.vtt`, `.srt`, `.mp4`, `.m4a` — any transcript or recording.
- **Paste.** Drop a Google Meet, Zoom, or Teams transcript directly.
- **Google Meet auto-ingest.** Connect your Google account, drop new recordings in a watched folder, they become briefs automatically.
- **Voice dial-in.** Call a Twilio number, talk to your AI product manager live via Gemini Live API.

Agents

- **BA Agent** — generates structured product briefs from transcripts. Runs optional AI Council for multi-model consensus.
- **Architect** — produces implementation-ready design from an approved brief: components, APIs, data model, risks, implementation order.
- **Dev Agent** — (optional local daemon) opens PRs from approved briefs using YOUR Claude Code / Cursor / Codex CLI.
- **QA Agent** — verifies tests pass before merge.
- **Memory Optimizer** — maintains Honcho-style persistent memory that grows with each interaction.

Integrations (in-app wizards, no config files)

Jira · Slack · GitHub · Linear · Notion · Google Chat · Google Drive · Google Docs · Google Meet · Microsoft Teams · Twilio · SendGrid · WorkOS SSO

Approvals

- Linear-inspired keyboard-driven queue (`J/K` navigate, `A` approve, `R` reject)
- Responsive: two-pane on desktop, single-pane + fixed bottom bar on mobile (thumb-reachable, safe-area-aware)
- Sticky action bar on brief detail pages
- Confidence bars, risk callouts, council reasoning inline
- Reply-to-approve via **email**, **Slack**, **WhatsApp**, and **SMS** — reply `APPROVE <token>` or `REJECT <token> <optional reason>` to a Workforce0 message and it's done

Ask-an-agent from chat

- `@ba status` / `@dev jobs` / `@qa status` / `@architect <id>` / `@supervisor` / `@help` from any **WhatsApp**, **SMS**, or **Slack channel**
- Zero-LLM-cost tier-1 responses (direct DB queries), tenant-scoped via `TeamMember.channelIds`

- Tier-3 vision (fully autonomous multi-agent group chat) scoped in `docs/plans/tier-3-autonomous-multi-agent-chat.md`

Infrastructure

- OpenTelemetry tracing (optional)
- Row-level tenant isolation via Prisma extensions
- Rate limiting + audit log + webhooks API
- Prometheus metrics endpoint
- Docker, Docker Compose, Railway, Render, Fly.io, DigitalOcean templates

---

Tech stack

LAYER	TECH
Frontend	Next.js 16 (App Router, Turbopack) · Tailwind v4 · Radix UI · Aperture design system
Backend	Fastify 5 · TypeScript 5.6 · Prisma 7 · PostgreSQL 15 · Redis 7 · BullMQ
AI	Gemini 2.0 Flash · Claude Sonnet 4.6 · GPT-o1 — routed through `ModelRegistryService`
Voice	Gemini Live API · Twilio Media Stream
Local agent	Node daemon · WebSocket to backend · invokes your Claude Code CLI
Auth	JWT + httpOnly cookies · WorkOS SSO · scoped-per-tenant localStorage
Testing	Vitest (1,349+ tests) · Playwright (E2E smoke)
Observability	OpenTelemetry · structured JSON logs via pino

Roadmap

- [x] Meeting ingestion (upload, paste, Google Meet, voice dial-in)
- [x] Multi-model AI Council
- [x] Jira, Slack, GitHub, Google Chat / Drive / Docs / Meet, Twilio, SendGrid, Teams, WorkOS SSO integrations
- [x] **Linear** and **Notion** integrations — both GA

- [x] In-app integration wizards (zero `.env` editing after initial keys)
- [x] First-run setup wizard
- [x] BYOK UI for Claude / Gemini / OpenAI
- [x] Local agent for PR generation via your CLI subscription
- [x] Aperture design system — Impact pass (saturated indigo, bolder type, deeper contrast)
- [x] **Mobile-friendly approval queue** — single-pane on phones, thumb-sized CTAs
- [x] **Reply-to-approve via WhatsApp + SMS** (same token pool as email)
- [x] **Slack Events API bidirectional** — approvals + `@mention` agent dispatch in channels
- [x] **WhatsApp `@mention` agent dispatch** — ask `@ba status`, `@dev jobs`, etc. from your phone
- [x] One-click Docker deploy
- [ ] Tier-3 autonomous multi-agent chat (agents drive the conversation, not just respond) — design doc
- [ ] Auto skill distillation (successful trajectories → proposed skills → admin review)
- [ ] Outcome-based skill ranking (feed OutcomeObserver results back into skill priority)
- [ ] Railway / Render / Fly.io one-click templates — configs shipped, not platform-verified yet
- [ ] Asana / Salesforce connectors
- [ ] Telegram per-agent bot (inline Approve/Reject buttons)
- [ ] Hosted demo on `demo.workforce0.dev`

See issues and discussions for what's in flight.

Contributing

We would love your help. Good first issues are tagged `good-first-issue`; integration PRs are always welcome.

- `CONTRIBUTING.md` — setup, coding standards, PR process
- **Catalogs** — the contributor's index for every pluggable piece of the app:
  - `docs/catalog/integrations.md` — all 16 connectors + add-a-new walkthrough
  - `docs/catalog/ai-providers.md` — Gemini/Claude/OpenAI + add-a-new walkthrough
  - `docs/catalog/agents.md` — BA, Architect, Dev, QA, Supervisor, Memory Optimizer + extend-an-agent walkthrough
- `AGENTS.md` — if you work with Claude Code, Cursor, or Codex on this repo
- `CODE_OF_CONDUCT.md` — be excellent to each other
- `SECURITY.md` — report a vulnerability privately

Repo layout

```
workforce0/
├── frontend/             Next.js 16 app (the UI you see in screenshots)
│   └── src/app/          App Router — (dashboard), (onboarding), login, signup, …
├── backend/              Fastify backend + Prisma + all agent services
│   ├── src/config/       Environment schema (zod)
│   ├── src/routes/       REST endpoints (auth, meetings, agents, settings, …)
│   ├── src/services/     BA agent, Architect, ModelRegistry, integrations, memory
│   ├── src/middleware/   Tenant isolation, rate limiting, auth
│   └── prisma/           Schema + migrations
├── agent/                Local daemon — runs on the user's machine, opens PRs
├── docs/                 Product + self-hosting guides, design system, BYOK
│   └── assets/           Logo, screenshots, diagrams
├── docker-compose.prod.yml   Full-stack deploy
└── .env.example          Root env for docker-compose.prod.yml
```

Security

- All user input validated with Zod at every route boundary
- Prisma parameterized queries prevent SQL injection
- React auto-escapes output; cookies are httpOnly + sameSite
- AES-256-GCM encryption for integration secrets and AI keys at rest
- Per-tenant row-level isolation via Prisma `$extends` middleware
- Agent daemon prompts for approval on every job; one `y` keystroke approves exactly one job (not N — serialized by design)
- See `SECURITY.md` to report vulnerabilities privately

License

MIT — free for personal and commercial use. Fork it, ship it, make it yours.

Built for teams who would rather make decisions than take notes.

⭐ If this is useful, leave a star. It helps non-technical execs discover the project.